UNIT – 5

ASSESSMENT – 1

PYTHON CODE

NIHARIKA PREEJESH

192524506

```
#
================================================================
========

# EXPERIMENT 1

# HYBRID AI RECOMMENDATION ENGINE USING PYSPARK
RDD

#
================================================================
========


from pyspark import SparkContext


sc = SparkContext("local[*]", "HybridRecommendation")


# ---------------------------------------------------------
# INPUT DATA
# user_id, item_id, rating
# ---------------------------------------------------------


ratings = [
    ("U1", "I1", 5),
    ("U1", "I2", 4),
```

```
    ("U1", "I3", 2),

    ("U2", "I1", 5),
    ("U2", "I2", 4),
    ("U2", "I4", 5),

    ("U3", "I1", 4),
    ("U3", "I3", 5),
    ("U3", "I5", 4),

    ("U4", "I2", 5),
    ("U4", "I4", 4),
    ("U4", "I5", 5)
]

ratings_rdd = sc.parallelize(ratings)

# ------------------------------------------------------
# ITEM CONTENT DATA
# item_id, category
# ------------------------------------------------------

items = [
    ("I1", "AI"),
```

```
    ("I2", "Python"),
    ("I3", "Web"),
    ("I4", "AI"),
    ("I5", "DataScience")
]

items_rdd = sc.parallelize(items)

print("USER-ITEM INTERACTIONS")
print("----------------------")

for record in ratings_rdd.collect():
    print(record)

# ---------------------------------------------------
# TARGET USER
# ---------------------------------------------------

target_user = "U1"

# ---------------------------------------------------
# CONTENT-BASED FILTERING
# Find categories liked by target user
# ---------------------------------------------------
```

```
user_ratings = ratings_rdd.filter(
    lambda x: x[0] == target_user and x[2] >= 4
)

liked_items = user_ratings.map(
    lambda x: x[1]
).collect()

liked_categories = items_rdd.filter(
    lambda x: x[0] in liked_items
).map(
    lambda x: x[1]
).collect()

content_recommendations = items_rdd.filter(
    lambda x: x[1] in liked_categories
).map(
    lambda x: x[0]
).collect()

print("\nCONTENT-BASED RECOMMENDATIONS")
print("-----------------------------")
print(content_recommendations)
```

```
# ------------------------------------------------------
# COLLABORATIVE FILTERING
# Find users having similar liked items
# ------------------------------------------------------

similar_users = ratings_rdd.filter(
    lambda x: x[1] in liked_items
).map(
    lambda x: x[0]
).distinct().filter(
    lambda x: x != target_user
)

similar_user_list = similar_users.collect()

print("\nSIMILAR USERS")
print("-----------------------------")
print(similar_user_list)

# ------------------------------------------------------
# Recommendations from similar users
# ------------------------------------------------------
```

```
collab_recommendations = ratings_rdd.filter(
    lambda x: x[0] in similar_user_list
    and x[2] >= 4
    and x[1] not in liked_items
).map(
    lambda x: x[1]
).distinct().collect()

print("\nCOLLABORATIVE RECOMMENDATIONS")
print("----------------------------")
print(collab_recommendations)

# ----------------------------------------------------
# HYBRID RECOMMENDATION
# ----------------------------------------------------

combined = list(
    set(content_recommendations + collab_recommendations)
)

print("\nFINAL HYBRID RECOMMENDATIONS")
print("----------------------------")

for item in combined:
```

```
    print(item)

# ------------------------------------------------------
# INTELLIGENT AGENT
# ------------------------------------------------------

print("\nINTELLIGENT AGENT")
print("----------------------------")

feedback = "like"

if feedback == "like":
    print("Agent detected positive feedback.")
    print("Agent increases priority of similar items.")

elif feedback == "dislike":
    print("Agent detected negative feedback.")
    print("Agent reduces priority of similar items.")

sc.stop()
```

```
#
================================================================
========
# EXPERIMENT 2
# SMART DISASTER DETECTION USING PYSPARK RDD
#
================================================================
========

from pyspark import SparkContext

sc = SparkContext("local[*]", "DisasterDetection")

# ----------------------------------------------------------
# SENSOR DATA
# location, disaster, intensity
# ----------------------------------------------------------

sensor_data = [
    ("ZoneA", "Flood", 8),
    ("ZoneB", "Earthquake", 7),
    ("ZoneC", "Flood", 3),
    ("ZoneA", "Flood", 9)
]
```

```
sensor_rdd = sc.parallelize(sensor_data)

# ----------------------------------------------------
# SATELLITE DATA
# ----------------------------------------------------

satellite_data = [
    ("ZoneA", "Flood", 9),
    ("ZoneB", "Earthquake", 6),
    ("ZoneC", "Flood", 2)
]

satellite_rdd = sc.parallelize(satellite_data)

# ----------------------------------------------------
# SOCIAL MEDIA DATA
# location, disaster
# ----------------------------------------------------

social_data = [
    ("ZoneA", "Flood"),
    ("ZoneA", "Flood"),
    ("ZoneB", "Earthquake"),
    ("ZoneC", "Flood")
```

```
]

social_rdd = sc.parallelize(social_data)

# ----------------------------------------------------
# DISPLAY INPUT
# ----------------------------------------------------

print("SENSOR DATA")
print("----------------")

for x in sensor_rdd.collect():
    print(x)

# ----------------------------------------------------
# HIGH INTENSITY SENSOR EVENTS
# ----------------------------------------------------

high_sensor = sensor_rdd.filter(
    lambda x: x[2] >= 7
)

print("\nHIGH INTENSITY SENSOR EVENTS")
print("----------------")
```

```
for x in high_sensor.collect():
    print(x)

# ------------------------------------------------------
# SATELLITE CONFIRMATION
# ------------------------------------------------------

satellite_confirmation = satellite_rdd.filter(
    lambda x: x[2] >= 7
)

# ------------------------------------------------------
# SOCIAL MEDIA COUNTS
# ------------------------------------------------------

social_count = social_rdd.map(
    lambda x: ((x[0], x[1]), 1)
).reduceByKey(
    lambda a, b: a + b
)

print("\nSOCIAL MEDIA REPORT COUNT")
print("----------------")
```

```
for x in social_count.collect():
    print(x)

# ----------------------------------------------------
# DATA FUSION
# ----------------------------------------------------

sensor_events = high_sensor.map(
    lambda x: ((x[0], x[1]), 1)
)

satellite_events = satellite_confirmation.map(
    lambda x: ((x[0], x[1]), 1)
)

combined = (
    sensor_events
    .union(satellite_events)
    .union(social_count)
)

evidence = combined.reduceByKey(
    lambda a, b: a + b
```

```
)

print("\nFUSED EVIDENCE")
print("---------------")

for x in evidence.collect():
    print(x)

# ------------------------------------------------------
# DISASTER DECISION
# ------------------------------------------------------

print("\nDISASTER ALERTS")
print("---------------")

alerts = evidence.filter(
    lambda x: x[1] >= 3
)

for location, count in alerts.collect():

    print(
        "ALERT:",
        location,
```

```
        "| Evidence:",
        count
    )

# ----------------------------------------------------
# INTELLIGENT AGENT
# ----------------------------------------------------

print("\nINTELLIGENT RESPONSE AGENT")
print("----------------")

for location, count in alerts.collect():

    print(
        "Agent response for",
        location,
        ": Issue early warning and recommend evacuation."
    )

sc.stop()
```

```
#
================================================================
========
# EXPERIMENT 3
# AI-BASED FRAUD DETECTION USING PYSPARK RDD
#
================================================================
========

from pyspark import SparkContext

sc = SparkContext("local[*]", "FraudDetection")

# ----------------------------------------------------------
# TRANSACTION DATA
# transaction_id, user, amount, location
# ----------------------------------------------------------

transactions = [
    ("T1", "U1", 500, "India"),
    ("T2", "U2", 1200, "India"),
    ("T3", "U1", 15000, "Unknown"),
    ("T4", "U3", 700, "India"),
    ("T5", "U2", 25000, "Unknown"),
    ("T6", "U4", 400, "India"),
```

```
    ("T7", "U1", 18000, "Unknown"),
    ("T8", "U3", 900, "India")
]

transaction_rdd = sc.parallelize(transactions)

# ----------------------------------------------------
# DISPLAY TRANSACTIONS
# ----------------------------------------------------

print("TRANSACTION DATA")
print("----------------")

for x in transaction_rdd.collect():
    print(x)

# ----------------------------------------------------
# ANOMALY DETECTION
# ----------------------------------------------------

# Transaction is suspicious if:
# amount > 10000 OR location is Unknown

suspicious = transaction_rdd.filter(
```

```
    lambda x:
        x[2] > 10000
        or x[3] == "Unknown"
)

print("\nSUSPICIOUS TRANSACTIONS")
print("----------------")

for x in suspicious.collect():
    print(x)

# ---------------------------------------------------------
# COUNT SUSPICIOUS TRANSACTIONS PER USER
# ---------------------------------------------------------

user_fraud_count = suspicious.map(
    lambda x: (x[1], 1)
).reduceByKey(
    lambda a, b: a + b
)

print("\nSUSPICIOUS TRANSACTIONS PER USER")
print("----------------")
```

```
for x in user_fraud_count.collect():
    print(x)

# ------------------------------------------------------
# INTELLIGENT FRAUD AGENT
# ------------------------------------------------------

print("\nFRAUD DETECTION AGENT")
print("----------------")

for user, count in user_fraud_count.collect():

    if count >= 2:

        print(
            "HIGH RISK USER:",
            user,
            "-> Account requires investigation."
        )

    else:

        print(
            "USER:",
```

```
            user,
            "-> Transaction flagged for review."
        )

sc.stop()



#
================================================================
========
# EXPERIMENT 4
# AUTONOMOUS SUPPLY CHAIN OPTIMIZATION
#
================================================================
========


from pyspark import SparkContext


sc = SparkContext("local[*]", "SupplyChain")


# ----------------------------------------------------------
# INVENTORY DATA
# product, current_stock
# ----------------------------------------------------------
```

```
inventory = [
    ("Laptop", 20),
    ("Phone", 50),
    ("Tablet", 15),
    ("Monitor", 8),
    ("Keyboard", 40)
]

inventory_rdd = sc.parallelize(inventory)

# ----------------------------------------------------
# DEMAND DATA
# product, expected_demand
# ----------------------------------------------------

demand = [
    ("Laptop", 35),
    ("Phone", 30),
    ("Tablet", 25),
    ("Monitor", 20),
    ("Keyboard", 15)
]

demand_rdd = sc.parallelize(demand)
```

```
# ------------------------------------------------------
# JOIN INVENTORY AND DEMAND
# ------------------------------------------------------

inventory_pair = inventory_rdd.mapValues(
    lambda x: ("Stock", x)
)

demand_pair = demand_rdd.mapValues(
    lambda x: ("Demand", x)
)

combined = (
    inventory_pair
    .union(demand_pair)
    .groupByKey()
)

# ------------------------------------------------------
# ANALYZE STOCK
# ------------------------------------------------------

print("SUPPLY CHAIN ANALYSIS")
```

```
print("----------------")

for product, values in combined.collect():

    stock = 0
    expected_demand = 0

    for value_type, value in values:

        if value_type == "Stock":
            stock = value

        elif value_type == "Demand":
            expected_demand = value

    shortage = expected_demand - stock

    print(
        product,
        "| Stock:", stock,
        "| Demand:", expected_demand,
        "| Shortage:", max(shortage, 0)
    )
```

```
# ----------------------------------------------------
# INTELLIGENT INVENTORY AGENT
# ----------------------------------------------------

print("\nINVENTORY AGENT")
print("----------------")

for product, values in combined.collect():

    stock = 0
    expected_demand = 0

    for value_type, value in values:

        if value_type == "Stock":
            stock = value

        elif value_type == "Demand":
            expected_demand = value

    if stock < expected_demand:

        order_quantity = expected_demand - stock
```

```
        print(
            product,
            "-> Restock",
            order_quantity,
            "units"
        )

    else:

        print(
            product,
            "-> Inventory sufficient"
        )

sc.stop()
```

```
#
================================================================
========
# EXPERIMENT 5
# PERSONALIZED LEARNING ANALYTICS
#
================================================================
========

from pyspark import SparkContext

sc = SparkContext("local[*]", "LearningAnalytics")

# ----------------------------------------------------------
# STUDENT DATA
# student, subject, score, study_hours
# ----------------------------------------------------------

student_data = [
    ("S1", "Python", 85, 8),
    ("S1", "Math", 55, 4),
    ("S1", "AI", 70, 6),

    ("S2", "Python", 60, 5),
    ("S2", "Math", 90, 9),
```

```
    ("S2", "AI", 80, 8),

    ("S3", "Python", 45, 3),
    ("S3", "Math", 65, 5),
    ("S3", "AI", 50, 4)
]

student_rdd = sc.parallelize(student_data)

# ----------------------------------------------------
# DISPLAY DATA
# ----------------------------------------------------

print("STUDENT LEARNING DATA")
print("----------------")

for x in student_rdd.collect():
    print(x)

# ----------------------------------------------------
# AVERAGE SCORE BY STUDENT
# ----------------------------------------------------

student_scores = student_rdd.map(
```

```
    lambda x: (
        x[0],
        (x[2], 1)
    )
).reduceByKey(
    lambda a, b: (
        a[0] + b[0],
        a[1] + b[1]
    )
)

print("\nAVERAGE STUDENT SCORE")
print("----------------")

for student, values in student_scores.collect():

    average = values[0] / values[1]

    print(
        student,
        "Average:",
        round(average, 2)
    )
```

```
# ------------------------------------------------------
# FIND WEAK TOPICS
# ------------------------------------------------------

weak_topics = student_rdd.filter(
    lambda x: x[2] < 60
)

print("\nWEAK TOPICS")
print("----------------")

for x in weak_topics.collect():

    print(
        x[0],
        "->",
        x[1],
        "| Score:",
        x[2]
    )

# ------------------------------------------------------
# INTELLIGENT LEARNING AGENT
# ------------------------------------------------------
```

```
print("\nLEARNING AGENT RECOMMENDATIONS")
print("----------------")

for student, subject, score, hours in weak_topics.collect():

    if score < 50:

        recommendation = (
            "Beginner lessons + practice quizzes"
        )

    else:

        recommendation = (
            "Revision videos + practice exercises"
        )

    print(
        student,
        "| Weak Subject:",
        subject,
        "| Recommendation:",
        recommendation
```

```
    )

# ------------------------------------------------------
# FEEDBACK
# ------------------------------------------------------

feedback = {
    "S1": "positive",
    "S2": "positive",
    "S3": "negative"
}

print("\nAGENT FEEDBACK ADAPTATION")
print("----------------")

for student, response in feedback.items():

    if response == "positive":

        print(
            student,
            "-> Continue current learning path"
        )
```

```
    else:

        print(
            student,
            "-> Agent recommends additional support"
        )

sc.stop()
```